

Technical report - Kampung Sambau

Track: Track 2B - Own Project. Event: Hack Apertus Online 2026.

Public project identifier: KampungSambau. Personal identities are omitted at the owner's request. Confirm the registered team alias before final submission.

Status: submission draft, 6 October 2026. Public repository URL and demo video URL are not yet available.

1. Summary

Kampung Sambau is a browser social-deduction game exploring character-specific interviews and embodied actions with an open language model. A player guards a village, compares testimony with identity cards and a resident register, and admits or rejects visitors. Apertus v1.5 70B generates English/German dialogue and selects turn/jump commands. Trusted game logic controls scoring and movement. This is an entertainment prototype, not real-world identity verification.

New hackathon work comprises the browser client, Docker packaging, Apertus action contract, validation, and USD-to-web animation conversion. The pre-existing owned NeighborhoodWatchVision project supplied characters, village assets, scenario data, visual design and the game concept. This reuse is disclosed; resolve eligibility with organizers if it conflicts with the requirement for a new hackathon project.

2. Architecture

The browser runs React, TypeScript and Three.js. A Node/Express service serves local GLB assets and UI textures, maintains private visitor profiles, dialogue history and scoring, and calls the configured chat-completions endpoint. Credentials are supplied through container environment variables and never included in browser bundles.

Data flows from browser text or WAV audio to the service, then Apertus, then a validated speech/action response and browser animation. Every response has a unique action ID and encounter ID. Rejection walks right outside a closed gate; admission walks left through the open gate. The renderer waits for departure completion before showing the next visitor.

Target architecture (mandatory)

Selected target: sovereign Swiss cloud. CSCS is the default inference endpoint. The application image can run behind an HTTPS reverse proxy on Swiss-operated infrastructure. This is a deployment design, not evidence of a completed sovereign deployment. Operator, jurisdiction, residency, application hosting and provider retention policies must be verified before final submission. No air-gapped operation or sovereign certification is claimed for the current local demo.

Build time uses a Node base image and locked npm dependencies. Offline asset conversion used OpenUSD, NumPy and Pillow; judges do not run it. Runtime needs the browser, application container and configured inference endpoint. There are no runtime asset CDNs, analytics services or implicit browser speech-recognition providers. A compatible on-premise Apertus endpoint is configurable, but its provisioning/hardware are not bundled or tested.

3. Use of Apertus

Model: swiss-ai/Apertus-v1.5-70B, nonthinking, through hosted CSCS inference. No fine-tuning or secondary runtime model is used. The prompt contains the character profile, card, selected language, roleplay constraints and recent structured history. It requests exactly a speech string and action enum: none, turn_around or jump.

Temperature is 0.35, dialogue limit 240 tokens, and request timeout 45 seconds. JSON-object response mode is requested; server validation permits one model repair attempt if output is malformed. The app never infers commands from prose or player keywords. Apertus selects actions; trusted game logic supplies physics,

including slower anomaly descent. Prompt and adapter sources are `src/server/game.ts` and `src/server/apertus.ts`.

Microphone recordings go to the same endpoint as WAV with a transcription instruction. Endpoint support and recognition quality, especially Swiss German, remain unverified. Players review a transcript before sending it. Text input remains available after audio errors.

4. Data

The included village dataset contains eight encounters and five resident profiles, with authored addresses, personalities and anomaly contradictions. The owner stated ownership of reused assets. Before publication, review names, portraits, texture text and any material derived from real people. No user audio or model responses are written to disk by the app. Sessions and history reside in memory with a nominal two-hour expiry checked on access.

The source game supplied scene/UI assets. Original USD walk deformation is exported to glTF morph animation at 12 frames per second. Source texture mappings appear in `public/assets/conversion-manifest.json`. The data directory is approximately 20 KB; make check enforces the template's 100 MB cap. No separate public Hugging Face dataset is currently proposed.

5. Evaluation

Verified locally: production/TypeScript build passed and nine automated tests passed. Tests cover strict response validation, repeated action IDs, encounter isolation, win/loss state, trusted jump timing, language prompting, rejection boundaries, animation presence in all eight GLBs, and actual Three.js vertex deformation during walk playback. These are engineering checks, not model accuracy measurements.

Browser inspection verified scene loading, rejection and progression to a second visitor without reported browser errors. Docker is unavailable on this development machine; no successful container run is claimed.

The previous baked-pose/jiggle implementation provides a qualitative animation baseline with no original walk clip. Current assets contain 33 sampled walk poses per visitor; the loader test confirms over 100 vertices change during playback while the animation root remains stationary. This is not a measurement of perceived animation quality.

Before submission, measure repeated English/German commands, negations, JSON validity, repair frequency, correct action rate, response latency, token usage and character consistency on the live endpoint. Swiss German requires a separate consented audio test set. No aggregate live metrics or cost claims are made without measurements.

6. Limitations

Unverified Docker execution, sovereign hosting, microphone compatibility and live-model reliability remain material gaps. Foliage conversion is imperfect; the idle animation loop is unported; sampled animation increases asset size. Sessions are not persistent or suitable for multi-instance deployment. Encounter order is random, so evaluations must record the encounter/request rather than assume a reproducible shuffle. Public-repo and demo links remain pending.

7. Reproducibility

On a clean checkout, configure `LLM_NAME`, `LLM_BASE_URL` and `LLM_API_KEY` in the environment or ignored `track_2b/.env`. Run `make run` from the repository root and open `http://localhost:3000`. The root forwards to the track directory. Docker Compose v2 and a WebGL2 browser are required. The app container needs no GPU with hosted inference. Initial builds fetch the Node base image and locked npm packages.

Docker builds run automated tests and the production build, then serve the app as a non-root user. `make down` stops it. Node 22 plus npm ci supports local development. `make check` validates layout/data limits. CI is

configured for a container smoke test with a dummy key, not a live inference test; no CI result is claimed yet. Record the actual implementation commit SHA for the final report. The current uncommitted changes must not be attributed to the template's commit.

8. Next steps

Validate Docker and live bilingual interactions, resolve audio capability, verify Swiss hosting, complete asset provenance/privacy review, publish a project repo, record a demo of at most two minutes, and regenerate the report with measured results and final identifiers.

License

Code: Apache-2.0. Documentation/report: CC-BY-4.0. Repository NOTICE describes provenance and remaining asset-level review. Required organizer registration details must still be supplied through the submission process.

References

- <https://hackapertus.notion.site/track-2-b-own-project>
- <https://github.com/HackApertus/project-template>
- <https://docs.cscs.ch/services/inference/api/>
- <https://huggingface.co/swiss-ai/Apertus-v1.5-70B>